

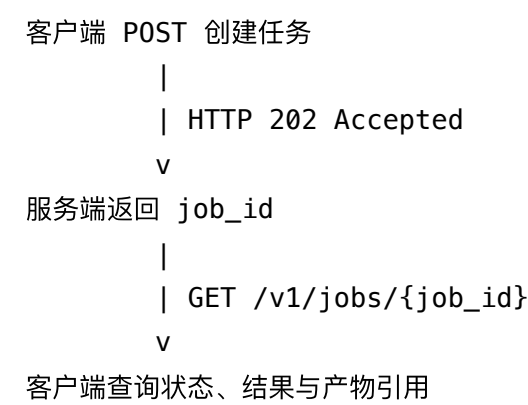
微服务接口定义文档

1. 总体接口风格

双方统一采用：

异步 Job + JSON 请求/响应 + Artifact 引用

基本调用流程：



1.1 通用请求头

```
Content-Type: application/json
Accept: application/json
Idempotency-Key: 2d6bf47d-d692-4ef0-b82c-e23dd3a375da
```

规则：

- 1. 所有创建任务的 POST 请求**必须**携带 Idempotency-Key 。
- 2. 同一服务收到相同幂等键和相同请求体时，应返回同一个 job_id ，不得重复创建任务。
- 3. 同一服务收到相同幂等键但请求体不同时，应返回 409 Conflict 。
- 4. job_id 由接收任务的服务端生成。
- 5. trace_id 由流程发起方提供，并在同一完整流程中保持不变。

1.2 参考端点

操作	方法与端点	成功状态码
生成构建环境	POST /v1/dockerfile-jobs	202 Accepted
全量检测	POST /v1/full-check-jobs	202 Accepted
增量检测	POST /v1/incremental-check-jobs	202 Accepted
修复缺失依赖	POST /v1/repair-jobs	202 Accepted
查询任务	GET /v1/jobs/{job_id}	200 OK

端点名称来自课程 PPT 的参考方案。如配对组修改端点名称，必须保持请求与响应语义不变，并在 ADR 中记录。

2. 统一 Job 模型

2.1 Job 对象

```
{
  "schema_version": "1.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "RUNNING",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": null
  },
  "input": {},
  "output": null,
  "error": null
}
```

2.2 公共字段

字段	类型	必填阶段	说明
schema_version	string	始终必填	契约版本，本版固定为 1.0
job_id	string	服务端响应必填	任务唯一编号，由服务端生成
trace_id	string	创建请求必填	串联一次完整 A/B 流程
job_type	enum	始终必填	DRAFT 、 FULL_CHECK 、 INCREMENTAL_CHECK 或 REPAIR
status	enum	服务端响应必填	当前任务状态
execution	object	服务端响应必填	任务创建、开始和结束时间
input	object	创建请求必填	服务专有输入，服务端查询时原样保留
output	object/null	查询响应必填	成功后的结果；未成功时为 null
error	object/null	查询响应必填	执行异常；无异常时为 null

2.3 job_type 枚举

DRAFT
FULL_CHECK
INCREMENTAL_CHECK
REPAIR

2.4 status 枚举与状态流转

QUEUED -> RUNNING -> SUCCEEDED
-> FAILED
-> TIMED_OUT
-> CANCELLED

状态	含义	是否终态
QUEUED	任务已受理，等待执行	否
RUNNING	任务正在执行	否

状态	含义	是否终态
SUCCEEDED	工具正常完成，结果可用	是
FAILED	工具执行失败	是
TIMED_OUT	任务超过时间限制	是
CANCELLED	任务被取消	是

重要约定：

检测到 MISSING 或 REDUNDANT 属于正常分析结果，任务状态仍为 SUCCEEDED 。只有工具没有正常完成时，任务才进入 FAILED 、 TIMED_OUT 或 CANCELLED 。

2.5 execution 对象

```
{
  "created_at": "2026-09-22T12:00:00Z",
  "started_at": "2026-09-22T12:00:03Z",
  "finished_at": "2026-09-22T12:03:18Z"
}
```

- 时间格式统一采用 UTC 的 RFC 3339 格式。
- 尚未开始时， started_at 为 null 。
- 尚未结束时， finished_at 为 null 。

3. 创建任务与查询任务

3.1 创建任务的统一响应

```
HTTP/1.1 202 Accepted
Content-Type: application/json
Location: /v1/jobs/job-full-001
```

```
{
  "schema_version": "1.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "QUEUED"
}
```

3.2 查询运行中的任务

GET /v1/jobs/job-full-001

```
{
  "schema_version": "1.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "RUNNING",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": null
  },
  "input": {},
  "output": null,
  "error": null
}
```

3.3 查询成功任务

```
{
  "schema_version": "1.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "SUCCEEDED",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": "2026-09-22T12:03:18Z"
  },
  "input": {},
  "output": {},
  "error": null
}
```

3.4 查询执行失败任务

```
{
  "schema_version": "1.0",
  "job_id": "job-full-001",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "status": "FAILED",
  "execution": {
    "created_at": "2026-09-22T12:00:00Z",
    "started_at": "2026-09-22T12:00:03Z",
    "finished_at": "2026-09-22T12:00:41Z"
  },
  "input": {},
  "output": null,
  "error": {
    "code": "ANALYSIS_5001",
    "message": "Analyzer failed",
    "details": {
      "log_uri": "artifact://pair03/full-001/analyzer.log"
    }
  }
}
```

4. Artifact 产物引用

Job 中只保存元数据和产物引用，不直接嵌入大日志、依赖图或 Patch。

4.1 ArtifactRef 结构

```
{
  "artifact_id": "graph-001",
  "type": "ACTUAL_GRAPH",
  "uri": "artifact://pair03/full-001/actual.json",
  "media_type": "application/json",
  "producer_job_id": "job-full-001",
  "commit": "0123456789abcdef0123456789abcdef01234567",
  "configuration_id": "cc-MODE0",
  "sha256": "8d969eef6ecad3c29a3a629280e686cff8ca..."
}
```

字段	是否必填	说明
artifact_id	是	产物唯一编号
type	是	产物类型
uri	是	双方约定可解析的产物地址
media_type	是	MIME 类型
producer_job_id	是	生成该产物的 Job
commit	依赖源码的产物必填	产物对应的完整提交 SHA
configuration_id	依赖构建配置的产物必填	产物对应的构建配置
sha256	建议	完整性校验值；样例中可省略

4.2 最小产物类型

DOCKERFILE
CONTAINER_IMAGE
BUILD_LOG
TEST_LOG
ANALYZER_LOG
ACTUAL_GRAPH
DECLARED_GRAPH
ERROR_REPORT
PATCH
REPAIR_REPORT

4.3 URI 读取约定

E2 阶段建议采用统一的逻辑 URI:

`artifact://{pair_id}/{job_id}/{file_name}`

例如:

`artifact://pair03/full-001/error-report.json`

双方必须额外约定一个共享 Artifact 根目录或下载接口, 使该 URI 能被实际读取。禁止交接以下本机绝对路径:

`/Users/某同学/Desktop/...`
`/home/某同学/project/...`
`C:\Users\某同学\Desktop\...`

最小联调时, 可以把:

`artifact://pair03/full-001/error-report.json`

映射为仓库内共享路径:

`artifacts/pair03/full-001/error-report.json`

A/B 双方需要证明下游能够读取该文件。

5. Repository、Build 与 Environment 公共对象

5.1 RepositoryRef

```
{  
  "url": "https://example.com/pair03/demo-project.git",  
  "commit": "0123456789abcdef0123456789abcdef01234567",  
  "project_root": "."  
}
```

规则：

- commit 必须是完整提交 SHA，不得只传分支名、标签或短 SHA。
- project_root 是相对仓库根目录的路径，默认值为 。

5.2 BuildSpec

```
{  
  "configuration_id": "cc-MODE0",  
  "clean_command": "make clean",  
  "build_command": "make",  
  "test_command": "make test",  
  "verify_command": "make test"  
}
```

- BuildChecker 至少需要 configuration_id 、 clean_command 和 build_command 。
- DRAFT 至少需要 build_command 和 verify_command 。
- MDFixer 至少需要构建、测试和重检所需命令。

5.3 EnvironmentRef

```
{  
  "dockerfile": {  
    "artifact_id": "dockerfile-001",  
    "type": "DOCKERFILE",  
    "uri": "artifact://pair03/draft-001/Dockerfile",  
    "media_type": "text/plain",  
    "producer_job_id": "job-draft-001",  
    "commit": "0123456789abcdef0123456789abcdef01234567"  
  },  
  "image_ref": "pair03/demo-project:0123456789ab"  
}
```

6. DRAFT 接口（B 组）

6.1 创建请求

POST /v1/dockerfile-jobs

Idempotency-Key: draft-pair03-001

Content-Type: application/json

```
{
  "schema_version": "1.0",
  "trace_id": "trace-pair03-001",
  "job_type": "DRAFT",
  "input": {
    "repository": {
      "url": "https://example.com/pair03/demo-project.git",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "project_root": "."
    },
    "build": {
      "build_command": "make",
      "verify_command": "make test"
    },
    "limits": {
      "max_iterations": 5,
      "timeout_seconds": 600
    }
  }
}
```

6.2 成功输出

```
{
  "dockerfile": {
    "artifact_id": "dockerfile-001",
    "type": "DOCKERFILE",
    "uri": "artifact://pair03/draft-001/Dockerfile",
    "media_type": "text/plain",
    "producer_job_id": "job-draft-001",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "image": {
    "artifact_id": "image-001",
    "type": "CONTAINER_IMAGE",
    "uri": "docker://pair03/demo-project:0123456789ab",
    "media_type": "application/vnd.oci.image.manifest.v1+json",
    "producer_job_id": "job-draft-001",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "iteration_logs": [
    {
      "artifact_id": "draft-log-001",
      "type": "BUILD_LOG",
      "uri": "artifact://pair03/draft-001/iteration-1.log",
      "media_type": "text/plain",
      "producer_job_id": "job-draft-001",
      "commit": "0123456789abcdef0123456789abcdef01234567"
    }
  ],
  "build_result": {
    "success": true,
    "exit_code": 0
  },
  "verify_result": {
    "success": true,
    "exit_code": 0
  }
}
```

7. FULL_CHECK 接口 (A 组)

7.1 创建请求

POST /v1/full-check-jobs

Idempotency-Key: full-pair03-001

Content-Type: application/json

```
{
  "schema_version": "1.0",
  "trace_id": "trace-pair03-001",
  "job_type": "FULL_CHECK",
  "input": {
    "repository": {
      "url": "https://example.com/pair03/demo-project.git",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "project_root": "."
    },
    "environment": {
      "dockerfile_uri": "artifact://pair03/draft-001/Dockerfile",
      "image_ref": "pair03/demo-project:0123456789ab"
    },
    "build": {
      "configuration_id": "cc-MODE0",
      "clean_command": "make clean",
      "build_command": "make"
    }
  }
}
```

7.2 成功输出

```
{
  "actual_graph": {
    "artifact_id": "actual-graph-001",
    "type": "ACTUAL_GRAPH",
    "uri": "artifact://pair03/full-001/actual.json",
    "media_type": "application/json",
    "producer_job_id": "job-full-001",
    "commit": "0123456789abcdef0123456789abcdef01234567",
    "configuration_id": "cc-MODE0"
  },
  "declared_graph": {
    "artifact_id": "declared-graph-001",
    "type": "DECLARED_GRAPH",
    "uri": "artifact://pair03/full-001/declared.json",
    "media_type": "application/json",
    "producer_job_id": "job-full-001",
    "commit": "0123456789abcdef0123456789abcdef01234567",
    "configuration_id": "cc-MODE0"
  },
  "error_report": {
    "artifact_id": "error-report-001",
    "type": "ERROR_REPORT",
    "uri": "artifact://pair03/full-001/error-report.json",
    "media_type": "application/json",
    "producer_job_id": "job-full-001",
    "commit": "0123456789abcdef0123456789abcdef01234567",
    "configuration_id": "cc-MODE0"
  },
  "summary": {
    "missing_count": 1,
    "redundant_count": 0
  }
}
```

8. INCREMENTAL_CHECK 接口 (A 组)

8.1 创建请求

POST /v1/incremental-check-jobs

Idempotency-Key: incremental-pair03-c1

Content-Type: application/json

```
{
  "schema_version": "1.0",
  "trace_id": "trace-pair03-001",
  "job_type": "INCREMENTAL_CHECK",
  "input": {
    "base_commit": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
    "repository": {
      "url": "https://example.com/pair03/demo-project.git",
      "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
      "project_root": "."
    },
    "environment": {
      "image_ref": "pair03/demo-project:bbbbbbbbbbbbbb"
    },
    "build": {
      "configuration_id": "cc-MODE0",
      "clean_command": "make clean",
      "build_command": "make"
    },
    "baseline": {
      "actual_graph_uri": "artifact://pair03/full-c0/actual.json",
      "declared_graph_uri": "artifact://pair03/full-c0/declared.json",
      "commit": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
      "configuration_id": "cc-MODE0"
    }
  }
}
```

校验规则:

1. baseline 必须存在。
2. baseline.commit 必须等于 base_commit。
3. baseline.configuration_id 必须等于当前 build.configuration_id。

4. repository.commit 必须是本次待检测的新提交。

8.2 成功输出

```
{
  "current_error_report": {
    "artifact_id": "error-report-c1",
    "type": "ERROR_REPORT",
    "uri": "artifact://pair03/incremental-c1/error-report.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0"
  },
  "finding_changes": {
    "introduced": ["finding-md-001"],
    "resolved": [],
    "unchanged": []
  },
  "updated_actual_graph": {
    "artifact_id": "actual-graph-c1",
    "type": "ACTUAL_GRAPH",
    "uri": "artifact://pair03/incremental-c1/actual.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0"
  },
  "updated_declared_graph": {
    "artifact_id": "declared-graph-c1",
    "type": "DECLARED_GRAPH",
    "uri": "artifact://pair03/incremental-c1/declared.json",
    "media_type": "application/json",
    "producer_job_id": "job-incremental-c1",
    "commit": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
    "configuration_id": "cc-MODE0"
  }
}
```

9. ERROR_REPORT 与 Finding

9.1 ERROR_REPORT 文件结构

```
{
  "schema_version": "1.0",
  "report_type": "ERROR_REPORT",
  "producer_job_id": "job-full-001",
  "repository": {
    "url": "https://example.com/pair03/demo-project.git",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "configuration_id": "cc-MODE0",
  "findings": [
    {
      "finding_id": "finding-md-001",
      "type": "MISSING",
      "target": "main.o",
      "dependency": "config.h",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "location": {
        "file": "Makefile",
        "line": 12,
        "column": 1
      },
      "evidence": {
        "summary": "main.o uses config.h but the dependency is not declared",
        "process": "cc -c main.c -o main.o",
        "observed_file": "config.h",
        "declaration_file": "Makefile"
      },
      "detector": "BUILD_CHECKER"
    }
  ]
}
```

9.2 Finding 字段

字段	必填	说明
finding_id	是	报告内稳定且唯一的发现编号

字段	必填	说明
type	是	MISSING 或 REDUNDANT
target	是	被检测的构建目标
dependency	是	缺失或冗余的依赖项
commit	是	发现对应的完整提交 SHA
location.file	是	声明所在文件，使用仓库相对路径
location.line	是	1 开始的行号
location.column	否	1 开始的列号
evidence	是	支持该结论的证据
detector	是	BUILD_CHECKER 、 E_CHECKER 或 INSTRUCTOR_ORACLE

9.3 Finding 类型

- MISSING = 实际需要，但声明缺失
- REDUNDANT = 已声明，但当前配置没有使用

MDFixer 只消费 MISSING 。如果报告同时包含 MD 与 RD，REPAIR 请求必须通过 finding_ids 明确选择 MD，不得把 RD 作为修复目标。

10. REPAIR 接口（B 组）

10.1 创建请求

```
POST /v1/repair-jobs
Idempotency-Key: repair-pair03-001
Content-Type: application/json
```

```
{
  "schema_version": "1.0",
  "trace_id": "trace-pair03-001",
  "job_type": "REPAIR",
  "input": {
    "repository": {
      "url": "https://example.com/pair03/demo-project.git",
      "commit": "0123456789abcdef0123456789abcdef01234567",
      "project_root": "."
    },
    "environment": {
      "image_ref": "pair03/demo-project:0123456789ab"
    },
    "error_report": {
      "uri": "artifact://pair03/full-001/error-report.json",
      "commit": "0123456789abcdef0123456789abcdef01234567"
    },
    "finding_ids": ["finding-md-001"],
    "build_file": "Makefile",
    "validation": {
      "build_command": "make",
      "test_command": "make test",
      "recheck_command": "./run-build-checker.sh"
    }
  }
}
```

接收前校验：

1. repository.commit 必须等于 error_report.commit 。
2. 报告正文中的 repository.commit 也必须与二者一致。
3. finding_ids 中的每一项必须存在于报告中。
4. 被选择的 Finding 必须全部为 MISSING 。
5. build_file 必须使用仓库相对路径。

10.2 成功输出

```
{
  "patch": {
    "artifact_id": "patch-001",
    "type": "PATCH",
    "uri": "artifact://pair03/repair-001/repair.patch",
    "media_type": "text/x-diff",
    "producer_job_id": "job-repair-001",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "repair_report": {
    "artifact_id": "repair-report-001",
    "type": "REPAIR_REPORT",
    "uri": "artifact://pair03/repair-001/repair-report.json",
    "media_type": "application/json",
    "producer_job_id": "job-repair-001",
    "commit": "0123456789abcdef0123456789abcdef01234567"
  },
  "declaration_style": "Preserved existing Makefile dependency declaration style",
  "validation": {
    "build": {
      "success": true,
      "exit_code": 0,
      "log_uri": "artifact://pair03/repair-001/build.log"
    },
    "test": {
      "success": true,
      "exit_code": 0,
      "log_uri": "artifact://pair03/repair-001/test.log"
    },
    "recheck": {
      "success": true,
      "remaining_selected_findings": []
    }
  }
}
```

MDFixer 产生候选 Patch 后，必须依次完成构建、测试与重检。候选无效时，不应把无效 Patch 作为成功结果交付；服务应拒绝该候选并在修复报告或任务错误中记录原因。

11. 错误模型

11.1 系统执行错误 job.error

系统错误发生在任务已被受理之后，写入 `job.error` 。

```
{
  "code": "ENV_3002",
  "message": "Image build failed",
  "details": {
    "log_uri": "artifact://pair03/draft-001/build.log"
  }
}
```

课程材料给出的公共错误码：

错误码	含义	建议终态
ENV_3002	镜像构建失败	FAILED
EXEC_4002	任务超时	TIMED_OUT
ANALYSIS_5001	分析器失败	FAILED

11.2 请求拒绝错误

请求在创建 Job 前校验失败时，不生成 `job_id` ，直接返回 HTTP 4xx：

```
{
  "schema_version": "1.0",
  "error": {
    "code": "VERSION_2001",
    "message": "Error report commit does not match repository commit",
    "field": "input.error_report.commit"
  }
}
```

错误码如下。这些编号是本配对组的契约建议，不是课程 PPT 的固定编号：

HTTP 状态	错误码	触发条件
400 Bad Request	REQUEST_1000	JSON 无法解析
422 Unprocessable Content	VALIDATION_1001	缺少必填字段或字段格式错误
422 Unprocessable Content	BASELINE_2001	增量检测缺少 baseline
422 Unprocessable Content	BASELINE_2002	baseline 提交或配置不匹配
422 Unprocessable Content	REPAIR_2001	REPAIR 选择了非 MISSING Finding
409 Conflict	VERSION_2001	报告 commit 与源码 commit 不一致
409 Conflict	IDEMPOTENCY_2002	相同幂等键对应不同请求体
404 Not Found	JOB_2004	job_id 不存在

12. 版本兼容性

12.1 可兼容变更

- 新增可选字段。
- 新增不影响既有语义的产物元数据。
- 在双方允许未知字段的前提下扩展对象。

12.2 破坏性变更

- 删除或重命名现有字段。
- 改变现有字段语义或类型。
- 改变状态、任务类型或 Finding 类型枚举。
- 将可选字段改为必填字段。

发生破坏性变更时，必须提升 schema_version 的主版本，并同步更新 Schema、样例、校验脚本和 ADR。

13. 最小验收用例

编号	用例	预期结果
T01	提交四类合法创建请求	均可表达，并返回 202 与 job_id

编号	用例	预期结果
T02	将 job_type 改为 ABC	返回 422 VALIDATION_1001
T03	删除增量请求的 baseline	返回 422 BASELINE_2001
T04	令 baseline commit 与 base commit 不一致	返回 422 BASELINE_2002
T05	REPAIR 选择 REDUNDANT Finding	返回 422 REPAIR_2001
T06	令报告 commit 与仓库 commit 不一致	返回 409 VERSION_2001
T07	检测正常完成并发现 MD	Job 为 SUCCEEDED , MD 写入 findings
T08	分析器崩溃	Job 为 FAILED , 错误为 ANALYSIS_5001
T09	下游读取 Artifact URI	能读取文件并核对 commit/configuration
T10	相同幂等键重复提交同一请求	返回同一个 job_id , 不重复执行